Unit III

Searching Algorithms

Prof. Archana Burujwale

Searching Algorithms

- Consider a database of banking system where information of all customers is stored, such as name, address and account number etc. If a manager wants to search for a record of a particular customer, he has to look for that record from among all records that has been stored in the database.
- This process of looking up for a particular record in a database is referred as **searching.**
- Searching Algorithms are designed to check for an element or retrieve an element from any data structure where it is stored. Based on the type of search operation, these algorithms are generally classified into two categories:

1. **Sequential Search**: In this, the list or array is traversed sequentially and every element is checked. For example: Linear search
2. **Interval Search**: These algorithms are specifically designed for searching in sorted data-structures. These type of searching algorithms are much more efficient than Linear Search as they repeatedly target the centre of the search structure and divide the search space in half. For Example: Binary search

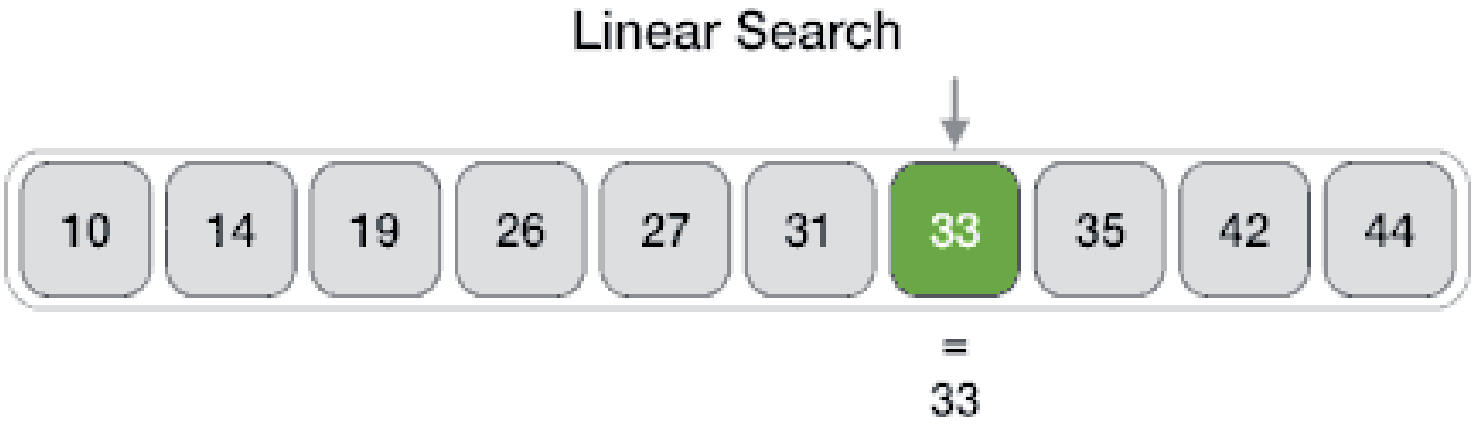
❖Searching Techniques are:

1. Sequential Search
2. Sentinel Search
3. Binary Search
4. Fibonacci Search
5. Indexed Sequential search

Sequential/Linear Search

- Linear search is a very simple search algorithm. In this type of search, a sequential search is made over all items one by one. Every item is checked and if a match is found then that particular item is returned, otherwise the search continues till the end of the data collection.
- A simple approach is to do a **linear search**, i.e

1. Start from the leftmost element of arr[] and one by one compare x with each element of arr[]
2. If x matches with an element, return the index.
3. If x doesn't match with any of elements, return -1.



Algorithm

Linear Search (Array A, Value x)

Step 1: Set i to 1
Step 2: if i > n then go to step 7
Step 3: if A[i] = x then go to step 6
Step 4: Set i to i + 1
Step 5: Go to Step 2
Step 6: Print Element x Found at index i and go to step 8
Step 7: Print element not found
Step 8: Exit

Pseudocode

```
procedure linear_search (list, value)
    for each item in the list
       if match item == value
         return the item's location
       end if
   end for
end procedure
```

- **Problem:** Given an array arr[] of n elements, write a function to search a given element x in arr[].
- **Examples :**

```
Input : arr[] = {10, 20, 80, 30, 60, 50,
                 110, 100, 130, 170}
          x = 110;
Output : 6
Element x is present at index 6

Input : arr[] = {10, 20, 80, 30, 60, 50,
                 110, 100, 130, 170}
           x = 175;
Output : -1
Element x is not present in arr[].
```

The **time complexity** of the above algorithm is O(n).

Sentinel Search

- It is a variant of sequential search.
- Sentinel Linear Search as the name suggests is a type of Linear search where the number of comparisons are reduced as compared to a traditional linear search.
- It is another way to reduce the overhead is to eliminate all checking of the loop index. This can be done by inserting the desired item itself as a sentinel value at the far end of the list
- When a linear search is performed on an array of size N then in the worst case a total of N comparisons are made when the element to be searched is compared to all the elements of the array and (N + 1) comparisons are made for the index of the element to be compared so that the index is not out of bounds of the array which can be reduced in a Sentinel Linear Search.
- In this search, the element to be searched is added at the end of the list and then the linear search is performed on the array without checking whether the current index is inside the index range of the array or not ,because the element to be searched will definitely be found inside the array even if it was not present in the original array since the last element is the search element only.
- So, the index to be checked will never be out of bounds of the array. The number of comparisons in the worst case here will be (N + 2).

- A linear search sequentially checks each element of the list until it finds an element that matches the target value. If the algorithm reaches the end of the list, the search terminates unsuccessfully
- **Basic algorithm**
- Given a list L of n elements with values or records L_0 .... L_{n-1}, and target value T, the following subroutine uses linear search to find the index of the target T in L.

1. Set i to 0.
2. If $L_i = T$, the search terminates successfully; return i.
3. Increase i by 1.
4. If $i < n$, go to step 2. Otherwise, the search terminates unsuccessfully.

- **With a sentinel**
- The basic algorithm above makes two comparisons per iteration: one to check if L_i equals T, and the other to check if i still points to a valid index of the list. By adding an extra record L_n to the list (a sentinel value) that equals the target, the second comparison can be eliminated until the end of the search, making the algorithm faster. The search will reach the sentinel if the target is not contained within the list.
- Set i to 0.
- If $L_i = T$, go to step 4.
- Increase i by 1 and go to step 2.
- If $i < n$, the search terminates successfully; return i. Else, the search terminates unsuccessfully.

Program for Sentinel Search

```
#include <stdio.h>

int sentinelLinearSearch(int arr[], int size, int target)
{
   // Storing the last element in temporary variable
   int last = arr[size - 1];
   // Setting the sentinel to the target
   arr[size - 1] = target;

   int i = 0;
   while (arr[i] != target) {
      i++;
   }

   // Restore the original value at the end
   arr[size - 1] = last;

   // checking if last element is target or not
   if (i < size - 1 || arr[size - 1] == target) {
      return i; // Element found at index i
   }
   else {
      return -1; // Element not found
   }
}
```

```
int main()
{
   int arr[] = { 3, 5, 1, 8, 2 };
   int size = sizeof(arr) / sizeof(arr[0]);
   int target = 8;

   int result = sentinelLinearSearch(arr, size, target);

if (result != -1) {
      printf("Element %d found at index %d\n", target,
           result);
   }
   else {
      printf("Element %d not found\n", target);
   }

   return 0;
}
```

Examples:

Input: arr[] = {10, 20, 180, 30, 60, 50, 110, 100, 70}, x = 180
Output: 180 is present at index 2

Input: arr[] = {10, 20, 180, 30, 60, 50, 110, 100, 70}, x = 90
Output: Not found

Binary Search

- The sequential search algorithm is very slow.
- If we have an array of 1000 elements, we must make 1000 comparisons in the worst case.
- The binary search starts by testing the data in the element at the middle of the array to determine if the target is in the first or the second half of the list.
- mid = (begin + end) / 2
- If it is in the first half, we do not need to check the second half.
- If it is in the second half, we do not need to test the first half.
- In other words, we eliminate half the list from further consideration with just one comparison. We repeat this process, eliminating half of the remaining list with each test, until we find the target or determine that it is not in the list.
- To find the middle of the list, we need three variables: one to identify the beginning of the list, one to identify the middle of the list, and one to identify the end of the list.
- We analyze two cases here: the target is in the list and the target is not in the list.

Binary Search



- **Binary Search:** Search a sorted array by repeatedly dividing the search interval in half. Begin with an interval covering the whole array. If the value of the search key is less than the item in the middle of the interval, narrow the interval to the lower half. Otherwise narrow it to the upper half. Repeatedly check until the value is found or the interval is empty.
- We basically ignore half of the elements just after one comparison.

1. Compare x with the middle element.
2. If x matches with middle element, we return the mid index.
3. Else If x is greater than the mid element, then x can only lie in right half sub array after the mid element. So we recur for right half.
4. Else (x is smaller) recur for the left half.

- The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(Log n)

How Binary Search Works?

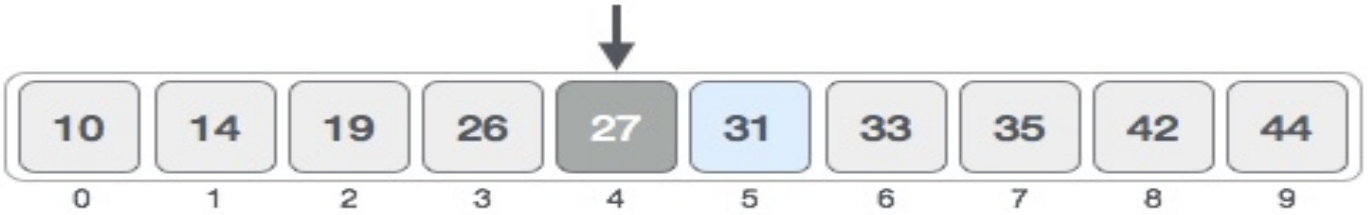
For a binary search to work, it is mandatory for the target array to be sorted. We shall learn the process of binary search with a pictorial example. The following is our sorted array and let us assume that we need to search the location of value 31 using binary search.



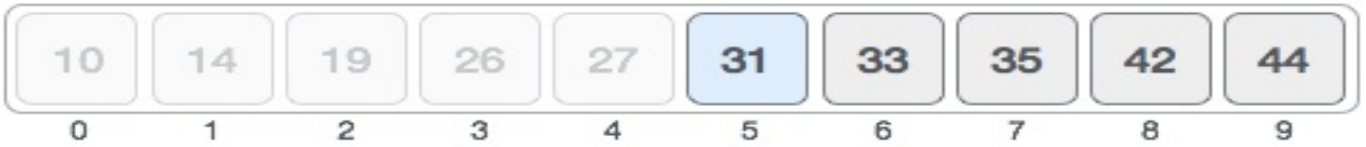
First, we shall determine half of the array by using this formula −

```
mid = low + (high - low) / 2
```

Here it is, 0 + (9 - 0) / 2 = 4 (integer value of 4.5). So, 4 is the mid of the array.



Now we compare the value stored at location 4, with the value being searched, i.e. 31. We find that the value at location 4 is 27, which is not a match. As the value is greater than 27 and we have a sorted array, so we also know that the target value must be in the upper portion of the array.



We change our low to mid + 1 and find the new mid value again.

```
low = mid + 1
mid = low + (high - low) / 2
```

Our new mid is 7 now. We compare the value stored at location 7 with our target value 31.



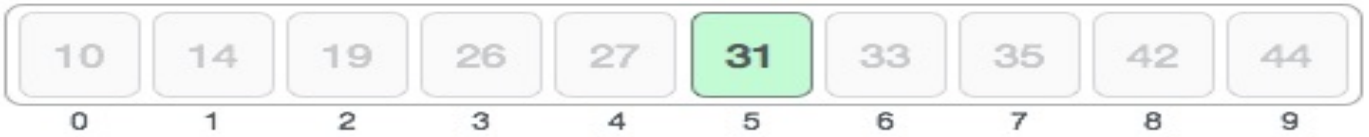
The value stored at location 7 is not a match, rather it is more than what we are looking for. So, the value must be in the lower part from this location.



Hence, we calculate the mid again. This time it is 5.



We compare the value stored at location 5 with our target value. We find that it is a match.



We conclude that the target value 31 is stored at location 5.

Binary search halves the searchable items and thus reduces the count of comparisons to be made to very less numbers.

Pseudocode

```
Procedure binary_search
   A ← sorted array
   n ← size of array
   x ← value to be searched

   Set lowerBound = 1
   Set upperBound = n

   while x not found
      if upperBound < lowerBound
         EXIT: x does not exists.

      set midPoint = lowerBound + ( upperBound - lowerBound ) / 2

      if A[midPoint] < x
         set lowerBound = midPoint + 1

      if A[midPoint] > x
         set upperBound = midPoint - 1

      if A[midPoint] = x
         EXIT: x found at location midPoint
   end while

end procedure
```

Fibonacci series

- a series of numbers in which each number (*Fibonacci number*) is the sum of the two preceding numbers.
- First few Fibonacci Numbers are 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, …

Fibonacci Search

- Fibonacci Search is a comparison-based technique that uses Fibonacci numbers to search an element in a sorted array.
- **Similarities with Binary Search:**
- Works for sorted arrays
- A Divide and Conquer Algorithm.
- Has Log n time complexity.
- **Differences with Binary Search**:
- Fibonacci Search divides given array in unequal parts
- Binary Search uses division operator to divide range. Fibonacci Search doesn't use /, but uses + and -. The division operator may be costly on some CPUs.
- Fibonacci Search examines relatively closer elements in subsequent steps. So when input array is big that cannot fit in CPU cache or even in RAM, Fibonacci Search can be useful.
- **Background:**
  Fibonacci Numbers are recursively defined as F(n) = F(n-1) + F(n-2), F(0) = 0, F(1) = 1. First few Fibonacci Numbers are 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, …

Fibonacci Search

Given a sorted array arr[] of size n and an element x to be searched in it. Return index of x if it is present in array else return -1.

Examples:

```
Input:  arr[] = {2, 3, 4, 10, 40}, x = 10
Output:  3
Element x is present at index 3.

Input:  arr[] = {2, 3, 4, 10, 40}, x = 11
Output:  -1
Element x is not present.
```

Algorithm:

Let the searched element be x.

- The idea is to first find the smallest Fibonacci number that is greater than or equal to the length of given array.
- Let the found Fibonacci number be fib (m'th Fibonacci number).
- We use (m-2)'th Fibonacci number as the index (If it is a valid index).
- Let (m-2)'th Fibonacci Number be i, we compare arr[i] with x, if x is same, we return i. Else if x is greater, we recur for subarray after i, else we recur for subarray before i.

- Below is the complete algorithm
  Let arr[0..n-1] be the input array and element to be searched be x.

1. Find the smallest Fibonacci Number greater than or equal to n. Let this number be fibM [m'th Fibonacci Number]. Let the two Fibonacci numbers preceding it be fibMm1 [(m-1)'th Fibonacci Number] and fibMm2 [(m-2)'th Fibonacci Number].

2. While the array has elements to be inspected:
    1. Compare x with the last element of the range covered by fibMm2
    2. **If** x matches, return index
    3. **Else If** x is less than the element, move the three Fibonacci variables two Fibonacci down, indicating elimination of approximately rear two-third of the remaining array.
    4. **Else** x is greater than the element, move the three Fibonacci variables one Fibonacci down. Reset offset to index. Together these indicate elimination of approximately front one-third of the remaining array.
3. Since there might be a single element remaining for comparison, check if fibMm1 is 1. If Yes, compare x with that remaining element. If match, return index.

Illustration:

Let us understand the algorithm with below example:

i	1	2	3	4	5	6	7	8	9	10	11	12	13
ar[i]	10	22	35	40	45	50	80	82	85	90	100	-	-

Illustration assumption:

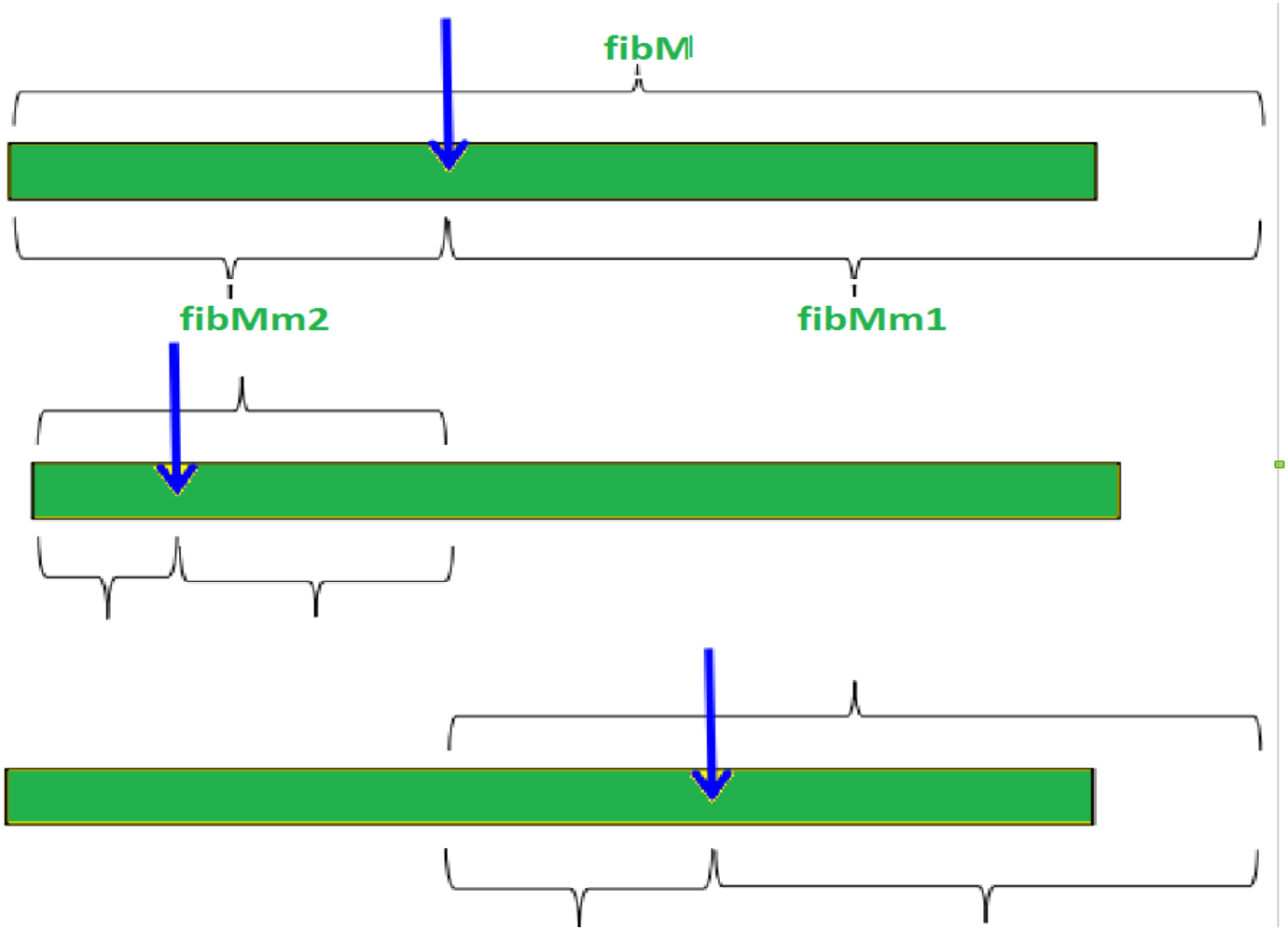
- 1-based indexing.
- Target element x is 85.
- Length of array n = 11.
- Smallest Fibonacci number greater than or equal to 11 is 13.
- As per our illustration, fibMm2 = 5, fibMm1 = 8, and fibM = 13.
- Another implementation detail is the offset variable (zero initialized).
- It marks the range that has been eliminated, starting from the front. We will update it time to time.

- Now since the offset value is an index and all indices including it and below it have been eliminated, it only makes sense to add something to it. Since fibMm2 marks approximately one-third of our array, as well as the indices it marks are sure to be valid ones, we can add fibMm2 to offset and check the element at index i = min(offset + fibMm2, n).

fibMm2	*fibMm1*	*fibM*	*offset*	i=min(offset+fibL n)	*arr[i]*	*Consequence*
5	8	13	0	5	45	Move one down, reset offset
3	5	8	5	8	82	Move one down, reset offset
2	3	5	8	10	90	Move two down
1	1	2	8	9	85	Return i

i	1	2	3	4	5	6	7	8	9	10	11	12	13
ar[i]	**10**	**22**	**35**	**40**	**45**	**50**	**80**	**82**	**85**	**90**	**100**	-	-

Visualization:



- **Observations:**
  Below observation is used for range elimination, and hence for the O(log(n)) complexity.

```
F(n - 2) &approx; (1/3)*F(n) and
F(n - 1) &approx; (2/3)*F(n).
```

- **Time Complexity analysis:**
  The worst case will occur when we have our target in the larger (2/3) fraction of the array, as we proceed to find it. In other words, we are eliminating the smaller (1/3) fraction of the array every time. We call once for n, then for(2/3) n, then for (4/9) n and henceforth.

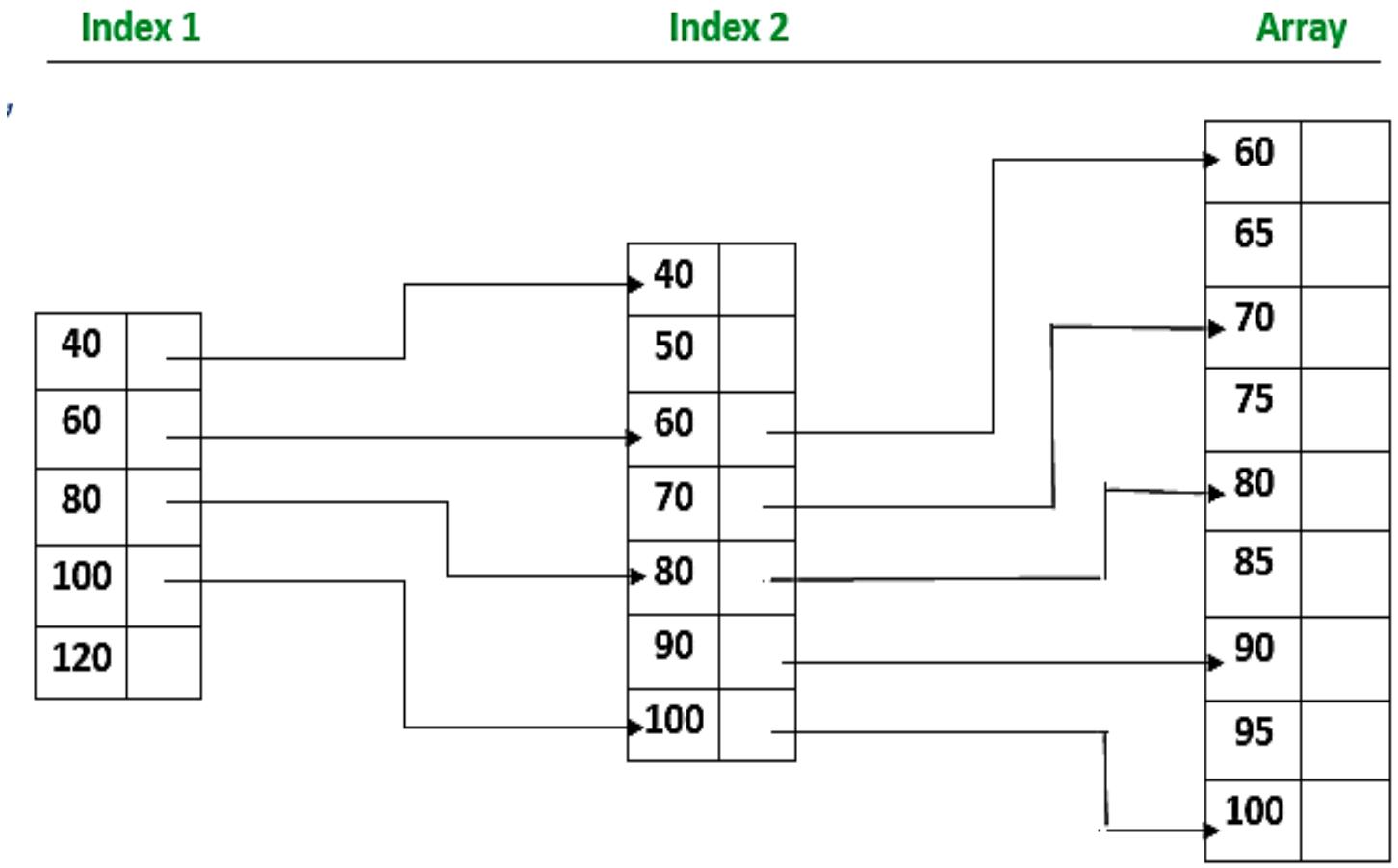
Indexed Sequential Search

- It is another method to improve the efficiency of searching in a sorted list. An auxiliary table called an index is set aside in addition to the sorted file itself.
- In this searching method, first of all, an index file is created, that contains some specific group or division of required record when the index is obtained, then the partial indexing takes less time cause it is located in a specified group.
- But it needs extra space; this method is also known as indexed sequential method.
- When the user makes a request for specific records it will find that index group first where that specific record is recorded.

- **Characteristics of Indexed Sequential Search:**
- In Indexed Sequential Search a sorted index is set aside in addition to the array. the partial searching takes less time since it is to be located in the group/bucket specified by the index.
- Each element in the index points to a block of elements in the array or another expanded index.
- The index is searched 1st then the array and guides the search in the array.
- Indexed Sequential Search actually does the indexing multiple time, like creating the index of an index.

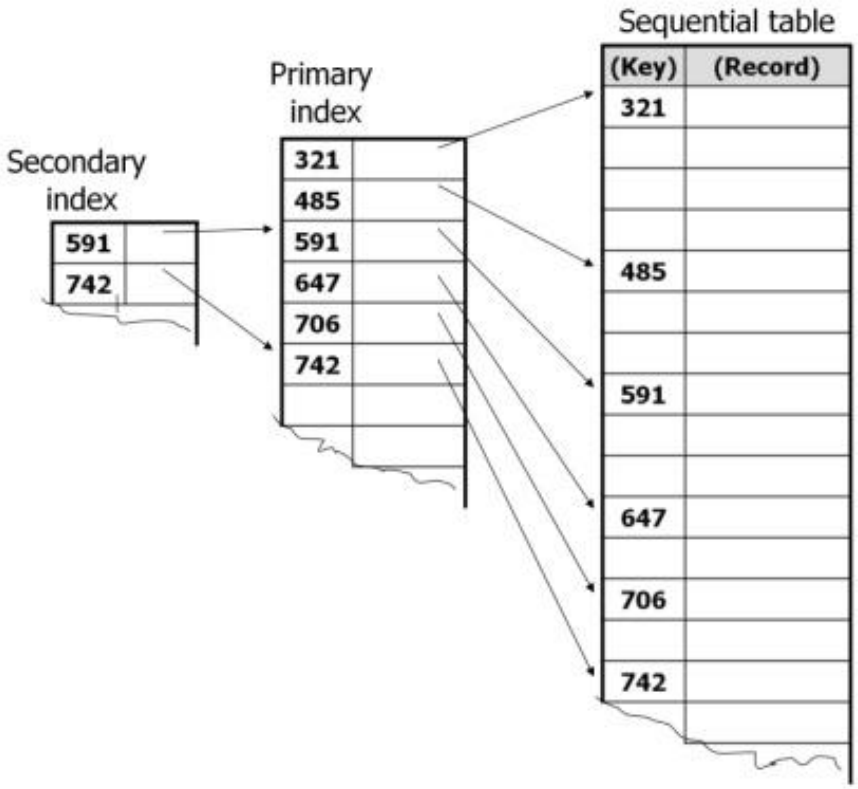
- For example program creates an index file for the employee records by grouping the records and then locates the required key by searching the index first and then returns the required record.
- The advantage of indexed sequential over sequential is that here we can achieve both sequential as well as Random access (i.e. jump directly to a record from the index value). Moreover, it is faster than sequential method.

Explanation by diagram “Indexed Sequential Search”:



• If the table is so large a single index cannot be sufficient to achieve efficiency. So, we can maintain secondary index.

•The secondary index acts as an index to the primary index. Which will points entry to the sequential table.



- **Indexed Sequential Search** is an optimized searching technique designed for sorted data arrays or files.
- It works by using an auxiliary data structure called an **index table** (or index file) alongside the primary data array to quickly isolate the exact segment (or block) where the target element resides, entirely bypassing the need to scan the entire data structure linearly from scratch.

Core Concept & Mechanism

- The algorithm functions similarly to looking up a term in a book's index
- **The Primary Array**: Must be sorted before constructing the index.
- **The Index Table**: Holds pairs of (Key, Reference Pointer/Index). It stores elements sampled at uniform intervals (e.g., every 3rd or 4th item) from the primary array along with their original indices

The Two-Step Search:

- **Step 1 (Index Scan)**: The algorithm performs a quick linear search through the small index table to locate the specific range/block where the target item falls.
- **Step 2 (Local Sequential Scan)**: Once the block boundary is identified, it switches to a traditional sequential search inside that small isolated block of the main array.

- **Step-by-Step Algorithm**
- Given a sorted array A of size n and a target item K:
- **Create the Index Table**: Choose a fixed block size b. Pick elements at intervals of b (i.e., 0, b, 2b, 3b...) and store their values and original positions into an auxiliary index table.
- **Scan the Index Table**: Sequentially traverse the index table to find a position i where Index_Table[i].Key <= K <Index_Table[i+1].Key.
- **Identify Boundaries**:
  - The local search starts at Start_Index = Index_Table[i].Position.
  - The local search ends at End_Index = Index_Table[i+1].Position (or the last index of the array if it's the final block).

  Perform Sequential Search: Iterate linearly from Start_Index to End_Index in the primary array A.
  - If A[j] == K, return j (Element Found).
  - If the loop finishes or A[j] > K, return -1 (Element Not Present).

Example:-

1. Setup Data Structures

- **Primary Sorted Array (`A`):**

Index	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**
Value	2	5	8	12	16	20	**25**	28

Constructed Index Table (Interval = 4):

Index Key	**Primary Position Reference**
2	0
16	4
32	8
48	12

- **2. Search Execution (Target = 25)**
- **Step 1: Check Index Table**
  - Is 25 between 2 and 16 ? No.
  - Is 25 between 16 and 32? **Yes**.
  - *Conclusion*: The target must lie in the block starting at primary position **4** (Value 16) and ending before primary position **8** (Value 32).
- **Step 2: Sequential Local Scan**
  We isolate our focus entirely to array index values 4 through 8:
  - Check Position 4: Value is 16 (Not a match).
  - Check Position 5: Value is 20 (Not a match).
  - Check Position 6: Value is **25** (**Match found!**).
  - **Result**: Element 25 successfully found at **Index 6**.

- **Complexity Analysis**
- **Time Complexity**:
  - **Best Case**: O(1) — If the element is located exactly at the first position of the index table.
  - **Average / Worst Case**: (O(n/b) + b) — where n/b represents scanning the index list and b represents sequentially checking the isolated block.
  - If block size b is chosen perfectly around sqrt(n), the time complexity scales down to **O(sqrt{n})**, which is vastly faster than a raw sequential linear search O(n)
  - **Space Complexity**: (**n/b)** — Auxiliary space is required exclusively to store the small index table.

```
// C program for Indexed Sequential Search

void indexedSequentialSearch(int arr[], int n, int k)
{
	int elements[20], indices[20], temp, i;
	int j = 0, ind = 0, start, end;
	for (i = 0; i < n; i += 3) {
		// Storing element
		elements[ind] = arr[i];
		// Storing the index
		indices[ind] = i;
		ind++;
		}
	 if (k < elements[0]) {
		printf("Not found");
		exit(0);
		}
		else {
		for (i = 1; i <= ind; i++)
		 if (k < elements[i]) {
			start = indices[i - 1];
			end = indices[i];
			 break;
		}
		}
for (i = start; i <= end; i++) {
				if (k == arr[i]) {
					j = 1;
					break;
				}
		}
		if (j == 1)
				printf("Found at index %d", i);
		else
				printf("Not found");
}

// Driver code
void main()
{

		int arr[] = { 6, 7, 8, 9, 10 };
		int n = sizeof(arr) / sizeof(arr[0]);

		// Element to search
		int k = 8;
		indexedSequentialSearch(arr, n, k);
}
```